Down the Lizard Hole

A deep dive into CR-Scan Lizard USB protocol reverse engineering

Matthew Armstrong (The-Dorkknight), with Claude (Anthropic) as lab assistant

October 2026 · version 1.0



> **Status: paused.** This project is resting for a month or two. Not because it's stuck, but because calibration turned out to be more boring than cleaning, and after weeks of chasing mad science projects the house has quietly filled up with jobs I've been ignoring. The mop wins this round. Everything here is written down so that anyone (including future me) can pick it up again.

In one paragraph

The Creality CR-Scan Lizard is a hand-held 3D scanner that normally only works with Creality's own Windows software. We wanted it to work on Linux with our own app. So we recorded every message between the scanner and Creality's app, and found something pleasantly boring: the Lizard is **two ordinary USB webcams behind a small USB hub**, plus **one small secret question** the app asks to read the scanner's serial number. There is no secret light switch: each light simply flashes while its own camera takes a picture, so **the length of the picture (the exposure) is the brightness control**. The infrared projector does not throw random dots, as we first thought, but **dense vertical stripes made of wobbly beads**. Once we matched those wobbles instead of looking for dots, a home-made "point it at a wall" calibration gave depth to within about **2 mm** on five test walls. Live depth works in parts and has known problems. This paper tells the whole story, wrong turns included, in plain English.

The app: Lizard Scanner (source-available, non-commercial). **The USB tool:** Who are yoUSB?.

---

1. Why go down the hole at all?

Like Alice, we followed something interesting without asking too many questions first.

The CR-Scan Lizard is a small, cheap structured-light 3D scanner. You wave it around an object and it builds a 3D model. Its own software runs on Windows (and macOS). We wanted three things:

1. **Run it on Linux**, on an ordinary laptop, without Creality's app.
2. **Control the lights** ourselves: the scanner has a white light ring and an infrared projector.
3. **Get depth** out of it, so it could plug into Scantastic, our own open scanning app (which already drives Intel RealSense and Kinect cameras).

Nobody had published how the Lizard talks over USB. One community guide had got Creality's older JM Studio app running under Wine on Linux, and to do it the author had to add two things to Wine: a pass-through for "vendor camera commands" and a bridge for raw USB. That was a hint that something special might be going on. As it turned out, for the app we recorded, almost nothing special was.

The rules we set ourselves

- **Listen first, never guess-write.** Reading from a device is safe. Writing random values to unknown vendor controls can corrupt a camera's firmware (the authors of the Linux infrared-emitter tool warn about exactly this). So we only ever sent what Creality's own app sent, or standard webcam settings.
- **No firmware changes, nothing written to the scanner.**
- **Keep the owner's serial number private.** It appears in this paper only as `CRSL*******`.
- **Not affiliated with Creality.** This is independent research for interoperability, done on a scanner we own.

2. What's in the box (as seen from USB)

Plug the Lizard into a computer and three USB devices appear:

USB ID	What it is	Notes
`1a40:0101`	A small USB 2 hub inside the scanner	The two cameras sit behind it

USB ID	What it is	Notes
`bcdc:000a`	“ICT CameraA”	A standard USB Video Class (UVC) webcam. The **infrared camera**; the infrared projector flashes with it
`bcdc:000b`	“ICT CameraB”	Another standard UVC webcam. The **white light ring** flashes with it

UVC is the standard every ordinary webcam uses, so Linux’s built-in `uvcvideo` driver picks both up with no help, and Creality’s own troubleshooting guide even tells users to test the scanner by opening “ICT CameraA” and “ICT CameraB” in a normal video player. Both run at USB 2 speed.

Creality’s published numbers: working distance **150 to 400 mm**, a single capture area of about **200 × 100 mm**, and “binocular” structured light (more on that word in section 9).

Each camera offers two video modes:

Mode	Size	Pictures per second
MJPEG (compressed)	1280 × 720	10
YUYV (uncompressed)	640 × 480	5

Each camera has 17 standard settings. The ones that matter: exposure from 1 to 120 (in units of 0.1 ms, so 0.1 ms to 12 ms; the camera starts at 60 = 6 ms), and gain from 16 to 255.

3. The tools

- **Who are yoUSB?**, our home-made USB “what is this and what is it saying?” toolkit. On Windows it records USB traffic with USBPcap (the recorder that comes with Wireshark); on Linux it uses the kernel’s `usbmon`. It then sorts every message into a readable report: which device, which request, which bytes, how often.
- **`v4l2-ctl`**, the standard Linux command for webcam settings and video.
- **Lizard Scanner**, the app this research produced. It has a Diagnostics window, a “Light test”, a “Gain test”, a “Sensor map” and a frame-capture panel that stamps every saved picture with its time, camera, exposure, gain and notes. Most of the measurements in sections 6 to 10 came from these.
- **A Windows virtual machine** running Creality’s CR Scan app, for the recordings.

4. Listening in: the two recordings (15 October 2026)

Recording 1: start the app, start scanning

We recorded about 55 seconds of USB traffic while CR Scan started, found the scanner and began a scan. Here is everything it said to the scanner that wasn't routine USB housekeeping, in order.

Step 1: "Who are you?" (the serial number read). About 19.6 seconds in, the app sends camera A a vendor command through what UVC calls an **extension unit**: a numbered box of manufacturer-specific controls. This is extension unit 4:

```
SET_CUR  unit 4, selector 10, 8 bytes:   02 c2 00 fe 14 00 00 00      (the question)
GET_CUR  unit 4, selector 11, 8 bytes:   00 00 00 10 43 52 53 4c      (answer, part 1)
GET_CUR  unit 4, selector 11, 8 bytes:   (seven serial digits, then 00)  (answer, part 2)
GET_CUR  unit 4, selector 11, 8 bytes:   ff ff ff ff bb bb bb bb      (answer, part 3: filler)
```

`43 52 53 4c` is ASCII for **"CRSI"**, the start of the Lizard's serial number (CR-Scan Lizard). The USB serial number the camera reports in the normal way is generic; this is the real one. The question looks like a general "read memory" command: `02` (read?), an address (`c2 00 fe`), and a length (`0x14` = 20 bytes). The answer is a little 16-byte record: a header `00 00 00 10` (0x10 = 16), then the 11-character serial. The app asks camera B the same question; camera B answers with nothing but `ff`.

Before each read the app asks `GET_LEN` ("how long is this control?") and gets `8`. Polite.

Step 2: "Show me pictures." About 27.6 seconds in, the app negotiates video with camera A using UVC's standard *probe / commit* handshake. The 26-byte messages are standard UVC, and they decode neatly:

Bytes in the message	Meaning
`01 01 40 42 0f 00`	format 1, frame size 1, frame interval `0x000F4240` = 1,000,000 × 100 ns = **0.1 s → 10 pictures a second**
`02 01 80 84 1e 00`	format 2, frame size 1, interval `0x001E8480` = 2,000,000 × 100 ns = **0.2 s → 5 a second**
max frame size `0x00096000` / `0x001C2000`	614,400 and 1,843,200 bytes: exactly 640×480 and 1280×720 pictures at 2 bytes a pixel

So the app briefly opens the small uncompressed mode, then settles on **MJPEG, 1280 × 720, 10 pictures a second**. Camera B never streamed in this recording.

Step 3: "Use these settings." Plain, standard UVC camera controls:

Control (standard UVC)	Read first	App sets
Auto-exposure mode (unit 1, selector 2)	1	**1** = manual

Control (standard UVC)	Read first	App sets
Exposure time (unit 1, selector 4), in 0.1 ms	60 (6 ms)	**10 (1 ms)**
Gain (unit 2, selector 4)	16	**65**

That's it. **No light commands.** No secret handshake. No calibration download from the scanner. The video was 26.8 MB in 27 seconds (about 1 MB/s), arriving as standard UVC packets of 1,536 bytes each, carrying JPEG pictures of 10 to 214 KB.

Recording 2: brightness, "sensitivity" and texture mode

The second recording had the owner moving every slider in CR Scan:

- The app's **brightness** slider changed only exposure and gain on camera A: exposure 1 ms → 0.5 ms, gain 65 → 40 → 30.
- The slider we thought was "focus" was actually **sensitivity**, and it too showed up as gain and exposure changes.
- Switching to **texture mode** (the white ring pulsed) only *restarted camera A's video* with the same sequence as before. No vendor command, and camera B was not used.
- When CR Scan quit, the lights went off.

What we concluded, and how much of it was right: the lights are not switched by any USB command; they follow the cameras' video. That part was right. We also concluded that camera A drives *both* lights. That part was wrong (section 6).

The complete "protocol"

For an app that wants to use the Lizard, the whole protocol fits on a sticky note:

1. Start camera A's video: standard UVC, MJPEG 1280 × 720 at 10 a second.
2. Set auto-exposure to manual, exposure to 10 (1 ms), gain to 65. Standard controls.
3. Optional: read the serial with extension unit 4 (selector 10 question, selector 11 answer).
4. For the white ring, start camera B the same way.

On Linux, steps 1 and 2 are just `v4l2-ctl` commands:

```
v4l2-ctl -d /dev/videoN --set-fmt-video=width=1280,height=720,pixelformat=MJPG
v4l2-ctl -d /dev/videoN --set-ctrl=auto_exposure=1,exposure_time_absolute=10,gain=65
```

5. The tea party of wrong ideas

Every rabbit hole has a mad tea party. Here are the confident ideas that turned out to be wrong, in the order we believed them. We keep them because the way they failed taught us more than the ideas did.

We believed	Because	It was wrong because
“The projector pulses every other picture”	CR Scan’s JPEGs alternated about 85 KB and 40 KB, and the red glow flickers	The projector flashes **once per picture**, so every picture is lit. The eye sees the 10-a-second flashing as flicker. Our pulse detector, built to find lit/dark pairs, kept saying “no pulse found” while the owner watched it flicker
“Camera A drives both lights”	In both recordings only camera A streamed, yet both lights were seen	The owner noticed the white ring lit only while **camera B** ran (with the infrared off). Each light belongs to its own camera. (Your lab assistant also told the owner, wrongly, that CR Scan only ever used camera A for everything, right after he’d said “cam 2” lit the ring. Noted and corrected.)
“Camera B is a colour texture camera”	Most structured-light scanners pair an infrared camera with a colour one	Camera B’s pictures are grey, and Lizard reviews describe greyscale texture. It may be a second infrared camera
“The calibration file is `ctrl_in_dev3_req81.bin`”	Who are yoUSB? saved a file in a folder called “calibration”	That file is just every answer the scanner gave to the app’s read requests: 41 bytes, mostly the serial. The real calibration was never seen on the wire (section 10)
“The pattern is random dots, like a Kinect”	Most cheap depth cameras use dots	It’s **vertical stripes made of beads** (section 8). This one cost us a whole failed calibration
“The pattern slides as one piece”	The textbook formula has a single number for the whole picture	The slide changes by about 25% from the left of the picture to the right (section 9)

6. Experiments with light

Once the app could start the cameras, the owner could watch the scanner’s front while we changed settings, and press “Yes, clearly”, “A little” or “Couldn’t tell”.

Exposure is the brightness control

The **Light test** steps one camera’s exposure from 0.1 ms to 1 ms, holding each step for 2.5 seconds, while you watch one light. Results (owner, 16 October 2026):

Run	Light watched	Owner saw	Picture brightness at 0.1 ms / 1 ms (0 to 255)
17:23	White ring (camera B)	Yes, clearly	0.2 / 17.3
17:24	White ring	Yes, clearly	0.0 / 17.2
17:26	White ring	Yes, clearly	1.1 / 79.7
17:31	White ring	Yes, clearly	48.9 / 252.5 (everything 10 to 15× brighter; cause unknown)

And the same for the infrared on camera A. Each light started half bright at 1 ms, dimmed at 0.1 ms and brightened again as the exposure stepped back up. So:

> **Each light flashes for as long as its own camera exposes.** A longer exposure is a longer flash, which looks brighter. No vendor command is needed to dim or brighten anything.

This also explains the owner's earlier report from CR Scan: with its brightness slider at maximum the white ring was "unbearably bright"; low, it was dim. That slider only ever changed exposure and gain.

Gain: probably just picture amplification

The **Gain test** holds the flash at 1 ms and steps the gain from 16 to 128. For both lights the owner answered "Couldn't tell". Gain is amplification inside the camera, and it can't sensibly drive an LED, so this fits. But "couldn't tell" isn't "no", so it's marked as open.

The slow drift

Twice, a step's pictures changed brightness while the setting stayed the same (79.7 vs 18.7 in the 17:26 run at 1 ms). Something in the scanner eases the light up slowly after a change. The owner described the same by eye: "starts half bright, then dims low, then very slowly ramps up". Our auto-tune now waits a few pictures before judging.

Camera B is dark at CR Scan's settings

At 1 ms and gain 65, camera B's picture averages only about 17 out of 255. Gain brightens it without lengthening the flash.

A safety cap

Because exposure *is* the flash, an app that lets a camera pick its own exposure can make the ring flash for 6 ms or more: six times longer than anything CR Scan ever asked for. Lizard Scanner therefore caps both cameras at CR Scan's own 1 ms. Brighter settings above that add gain only. Longer flashes (up to the camera's own 6 ms starting value, never more) sit behind a switch with a warning that turns itself off when the app closes.

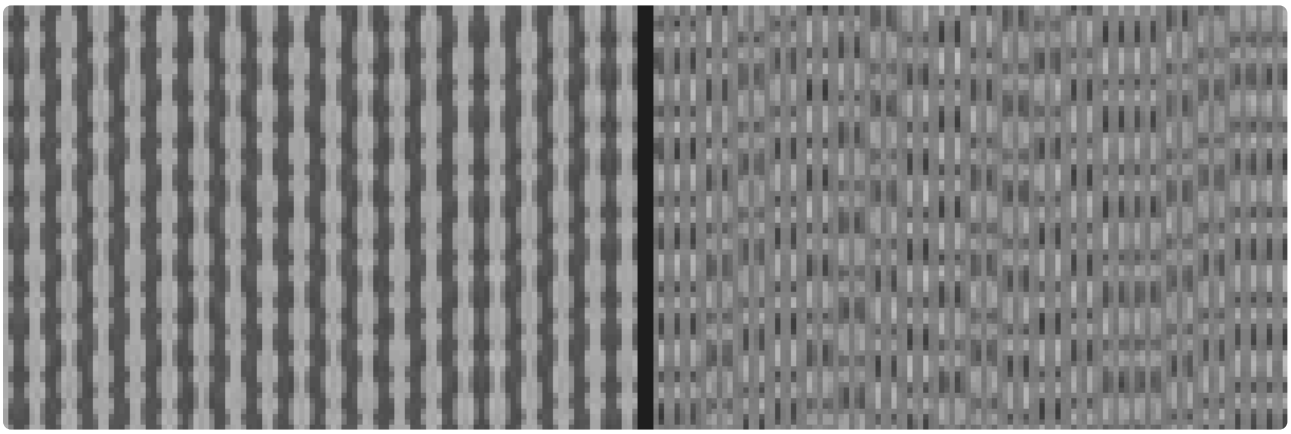
7. The Sensor map: who does what

The **Sensor map** runs camera A alone, camera B alone, then both, three seconds each, measures each camera's pictures, and asks which light you saw. What we know now:

	Camera A (`bcdc:000a`)	**Camera B (`bcdc:000b`)**
Its light	Infrared projector (red glow)	White ring
Picture	Grey infrared, shows the projector's pattern	Grey; shows a picture only while the white flash fires
Rate	10 a second (MJPEG 1280 × 720)	Lower with the ring on (seen earlier in Who are yoUSB?)
Answers the serial question	Yes	All `ff`
Used by CR Scan in our recordings	Yes	No

Covering camera A made everything go dark, so some of the earlier "two camera windows" in Who are yoUSB? may both have been camera A. Not confirmed.

8. The pattern: stripes, not dots



The projector's pattern on a flat wall (left, 120 × 80 pixels of the 640-wide picture, enlarged 4×) and its "stripe code" (right): what's left when the plain stripes are taken away.

The first real wall test (section 9) failed, and the Depth view filled with red and green noise. The owner put it best: *"the IR laser projects vertical lines with staggered dots."* His photos and saved pictures showed:

- **Dense vertical stripes**, about **6.34 pixels apart** at the 640-pixel-wide working size (about 12.7 at the camera's full 1280).
- Each stripe is a string of **beads**. Their width and sideways position wobble irregularly along the stripe, differently on every stripe. Those are the "staggered dots".

Why dots-thinking failed

A random dot pattern is easy to match: every small patch looks different from every other. Stripes are the opposite. Slide a picture of stripes sideways by one stripe spacing (6.3 px) and it looks almost the same. Our dot matcher used small square windows, and on the owner's walls the best match beat the second-best by only about **0.03** (on a scale where 1 is a perfect match): a coin toss. It picked the right stripe only 2 to 6 times in 10.

The trick: match the wobble, not the stripes

The plain stripe is the same all the way up. The wobble isn't. So for every pixel we subtract the average of the 17 pixels straight above and below it. What's left is only the wobble: the **stripe code** (right-hand picture above). Then we match tall, narrow windows of it (49 pixels tall, 21 wide at 640 × 360), which cover many beads along a few stripes. The best match now beats every rival by about **0.25**: a clear winner.

Is this a known kind of pattern?

There are three big families of patterns for structured-light scanners:

1. **Random dots ("speckle").** The original Kinect, built on PrimeSense's design, Intel's RealSense D400 and many cheap scanners. Every patch is unique, so one picture gives depth.
2. **Fringes.** Smooth sine-wave stripes, shifted a few times in a row. Very accurate, but needs several pictures per depth map and a still object.
3. **Coded lines.** Stripes that carry a code *along* them (cuts, dots, wobbles or widths), so you can tell one stripe from its neighbours in a single picture.

The Lizard's pattern belongs to family 3. The closest published idea we know is **"multiple slits with random cuts"** (Maruyama and Abe, 1993): parallel lines of light broken at random points, so each piece of line can be identified. Related schemes give the stripes different widths or colours following a **De Bruijn sequence** (a sequence where every short run appears only once). Salvi, Pagès and Batlle's survey and Geng's tutorial (see References) describe them all.

Important: Creality has never said which pattern the Lizard uses, so "family 3" is our reading of the pictures, not a specification. If the beads are a designed code rather than random, a decoder could read which stripe is which directly. That would be faster and better at edges. To find out, you'd need a sharp close-up of the pattern and some patience.

9. Measuring depth with a wall and a tape measure

The idea in one picture

The camera sits a little to one side of the projector. That gap is the **baseline**: about 6 cm on the Lizard, by our estimate. A stripe that lands on something near the scanner

appears shifted sideways in the camera's picture, compared with the same stripe landing on something far away. Measure how far the stripes shift at a few known distances, and every picture afterwards can be turned into distance.

For a perfect camera and projector, the shift between a wall at distance Z and a reference wall at Z_{ref} is

> $\text{shift} = K \times \left(1 / (Z + c) - 1 / (Z_{\text{ref}} + c) \right)$

where **K** is the camera's focal length (in pixels) times the baseline (in mm), and **c** says where the lens really sits compared with wherever you measured from.

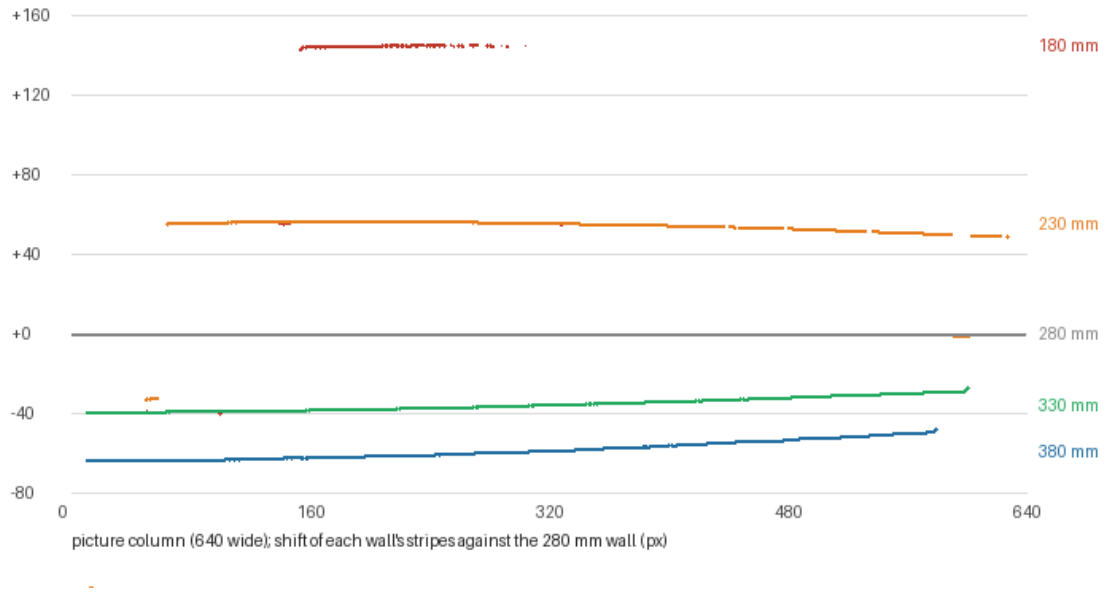
The wall test

1. Point the scanner straight at a flat, plain wall.
2. Measure the distance; type it in.
3. Record. The app averages six pictures into one "pattern picture" and keeps three of the camera's own full-size JPEGs.
4. Repeat at 18, 23, 28, 33 and 38 cm.

The owner measured from a point he chose (the lens is set back behind the front glass) and used the same point every time. That's fine: the fitted *c* absorbs it, and depth is then counted from that point.

What five walls taught us

Each wall is matched to its neighbour (180 against 230 mm, 230 against 280 mm and so on), at every point of the picture, and the steps are chained back to the middle wall (280 mm):



How far each wall's stripes moved against the 280 mm wall, across the picture.

- The shift maps are smooth: neighbouring walls moved 89, 56, 39 and 24 pixels at the left edge.
- **The shift changes across the picture**: 56 px at the left to 51 px at the right, between 230 and 280
  mm. So K isn't one number. It varies by about 25% (39,000 to 52,000 px·mm), probably from lens distortion and a projector that isn't perfectly square. The calibration stores a **K map**.
- The 180 mm wall moves about 145 px against the reference, so the left part of its picture has no partner in the reference at all. Only 34% of it matches.

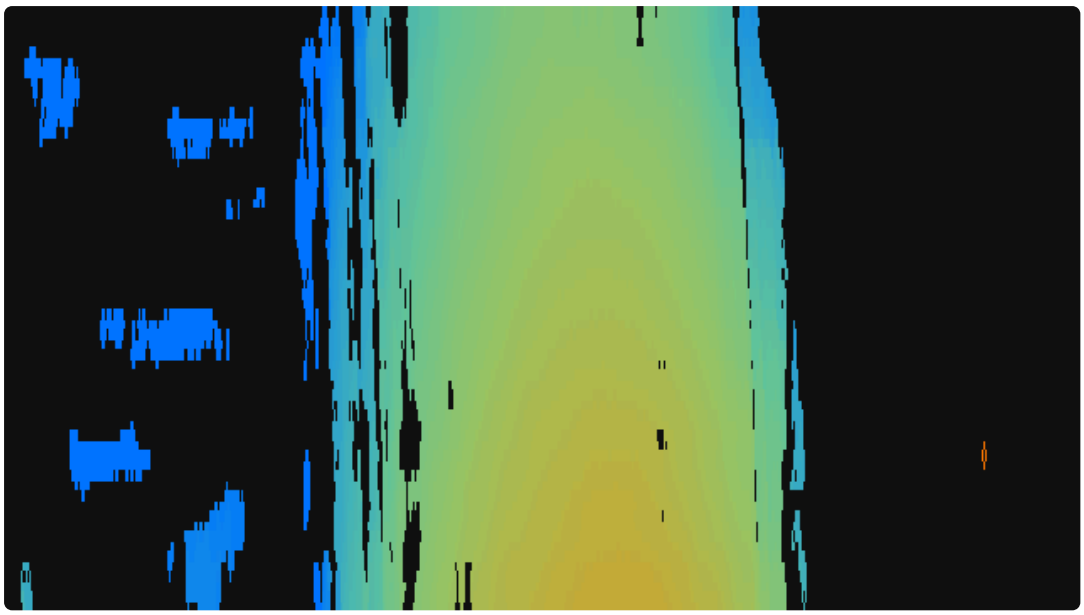
The fit

Wall (mm)	Matched	Shift (px)	Error (mm)	Spread (mm)	Right minus left (mm)
180	34%	143.2	+0.7	0.7	−1.2
230	70%	55.8	−1.6	0.8	−2.5
280	reference				
330	86%	−36.3	+1.9	1.8	−3.9
380	81%	−59.9	−1.6	2.3	−5.2

Every wall comes back within 2 mm of the tape measure. Every wall is also deeper on its left, more so further away. That's what you get when the scanner is turned slightly to the wall, about **1.7 degrees** here. The fitted c came out as 42 mm in front of the measuring point, which isn't physical. With only five walls, K and c can trade off against each other, and a millimetre of tape error moves c a lot. What matters is that depth inside the measured range is right.

The baseline, roughly: with the focal length guessed at about 830 px (from Creality's 200 mm capture width), a typical K of 48,000 px·mm means a baseline of about **58 mm**.

A mug



Depth of a mug held about 30 cm away (warm = nearer, 280 to 320 mm; dark = no depth).

From one saved picture of a mug held in front of the scanner: the mug's face comes out about 294 mm away in the middle and 301 to 312 mm towards its sides, a curved surface, as it should be. The hand holding it was closer than the working range, and it got no depth.

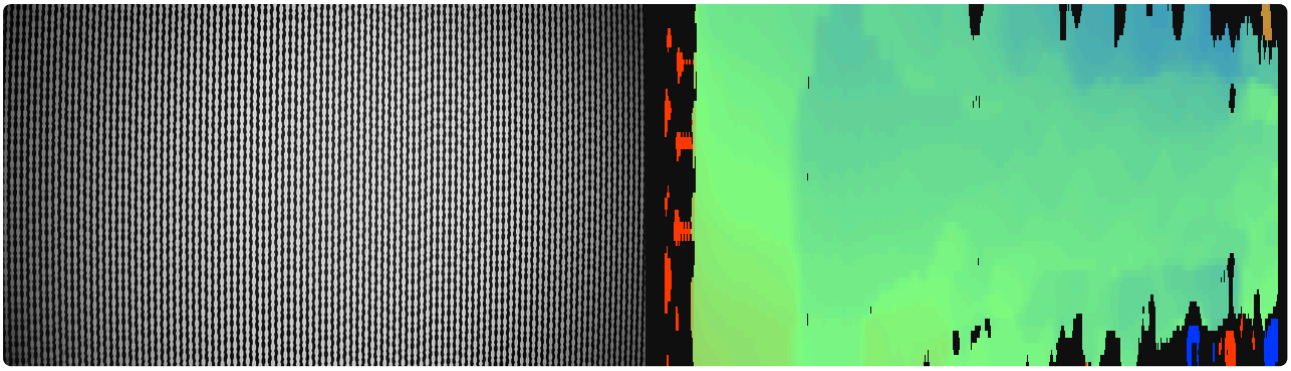
10. The live test, and why it's paused

19 October: green with red stripes

The owner recorded a fresh calibration and pointed the scanner at a wall about 25 cm away. The Depth view was **mostly green with red vertical streaks**. Objects showed up as a few green specks. The flat- wall check found depth in **0%** of the view. The diagnostics report said:

- Camera A: 10.1 pictures a second, the stripe pattern in 100% of the view, very little room infrared.
- Speed on the owner's laptop (Intel i5-8350U): **3.1 seconds** for a full depth search, 0.49 s while tracking a steady scene. That's about six times slower than on our build machine.
- The new calibration *looked* fine (every wall within 2.3 mm), but each wall matched only 50 to 70% of its neighbour, compared with 70 to 86% the day before.

The report included one live picture. Run through **the previous day's calibration**, it comes out as a flat wall at **240.7 mm** over 84 to 95% of the view:



The live picture from the report (left) and its depth with the 18 October calibration (right). Most of it is one flat wall; the streaks at the top right and bottom right are matches on the wrong stripe.

Three findings:

1. **The red streaks are wrong-stripe matches.** Matching on the neighbouring stripe puts a point about 6 px out, which is a big distance error. And because the matching windows are tall and narrow, the error shows up as a thin vertical streak. Version 0.5.1 checks each point against its neighbours and drops the ones that disagree.
2. **The 0% was a bug in the check**, not the depth: it kept only pixels that had depth in *all three* of its pictures, and the holes move between pictures.
3. **Why the new calibration does worse live is still unknown.** Its walls weren't in the report.

The real reason it's paused

The next steps are all calibration: record walls more carefully, compare two calibrations, speed up the search for slower laptops. That's careful, repetitive work, and the owner's verdict was honest: calibration is more boring than cleaning, and the cleaning has been piling up. So the lizard goes back into its hole for a month or two. The notes below are for whoever comes back to it.

11. Where's Creality's own calibration?

Every structured-light scanner needs a per-unit calibration, and the Lizard's real one would beat our wall test. We never saw it on the wire, but neither recording was a scanner's first-ever connection, so a file read once and saved would not show up. It can only be in three places:

1. **On the scanner**, read over USB maybe once. The "read memory" command that fetches the serial could in principle read more, but only replaying it with other addresses would tell, and that's left until last.
2. **On Creality's server**, downloaded by serial number.
3. **Cached on disk** by the app after either of those.

The plan is in the app repo (`docs/research/calibration-hunt.md`): search the Windows install for files that mention the serial, then record a "first connect" (USB, network and

file activity at once) after making the app forget the scanner.

12. Still open: a to-do list for brave readers

- Live depth on the real scanner, with a good calibration.
- **Why the second calibration was worse.** Its walls are needed; the app's ZIP CALIBRATION button makes a file of them with the serial hidden.
- **Speed** on older laptops: the full search is slow; tracking helps; a stripe decoder would help more.
- **Is the bead pattern a designed code?** If so, decode it.
- **Is camera B a second infrared camera?** Creality calls the Lizard "binocular". If camera B sees the projector's pattern too, the Lizard might be an *active stereo* scanner (two cameras plus a pattern), and the depth method could change completely.
- **Does gain change either light?** (A dim-room re-run of the gain test.)
- What made the 17:31 light-test run ten times brighter.
- A full-size (1280 × 720) calibration from the saved wall JPEGs.
- Creality's own calibration (section 11).

13. What we learned, in plain words

1. **Look for the boring answer first.** A "3D scanner" turned out to be two standard webcams. 95% of the protocol is the USB Video Class, which has been documented for years.
2. **The one vendor command was a question, not a switch.** It reads the serial.
3. **The lights follow the shutter.** No light commands at all: exposure *is* brightness.
4. **Write down every assumption, and when it falls, say so.** Six fell in five days, and each fall taught something.
5. **Ask the person holding the device.** "Cam 2 lights the ring", "it's lines with staggered dots", "starts half bright then slowly ramps up": each beat a day of guessing.
6. **Simulators lie politely.** Our pretend scanner projected dots because we told it to, so every test passed while the real scanner disagreed.

Appendix A: USB cheat sheet

Item	Value
Hub	`1a40:0101` (USB 2)
Camera A	`bcdc:000a`, "ICT CameraA", UVC, infrared + projector
Camera B	`bcdc:000b`, "ICT CameraB", UVC, white ring

Item	Value
Video (CR Scan)	MJPEG 1280 × 720 at 10/s (UVC format 1, interval 1,000,000 × 100 ns); also YUYV 640 × 480 at 5/s
Exposure	UVC camera terminal (unit 1), selector 4 (exposure time absolute), units of 0.1 ms; CR Scan: 10
Auto-exposure mode	unit 1, selector 2; CR Scan: 1 (manual)
Gain	processing unit (unit 2), selector 4; CR Scan: 65 (brightness slider: 40, 30)
Serial question	extension unit 4, selector 10, `SET_CUR` 8 bytes `02 c2 00 fe 14 00 00 00`
Serial answer	extension unit 4, selector 11, three `GET_CUR` of 8 bytes: `00 00 00 10` + `CRSL` + 7 digits + `00` + `ff ff ff ff bb bb bb bb`
Camera B’s answer	all `ff`
Video packets	1,536 bytes each, standard UVC payload headers
Light commands	none seen; each light flashes during its own camera’s exposure

Appendix B: the pattern and the wall test in numbers

Thing	Value
Stripe spacing	6.34 px at 640 wide (≈ 12.7 at 1280)
Stripe-code running mean	17 rows (8 each side)
Matching window	49 × 21 px at 640 × 360
Match accepted	score ≥ 0.30, beats every alignment more than 3 px away by ≥ 0.06, agrees with its neighbours within 1.5 px
Typical K	39,000 to 52,000 px·mm across the picture
Baseline (if focal guess is right)	≈ 58 mm
Wall errors (18 October)	within 2 mm at 180 to 380 mm
Full depth search	0.55 s (build machine), 3.1 s (i5-8350U laptop)
Tracking	0.14 to 0.29 s (build machine), 0.49 s (laptop)

Appendix C: timeline

Date (2026)	What happened
15 Oct	Two USB recordings of CR Scan. Serial read found; lights not switched by any command; “pulses every other picture” believed

Date (2026)	What happened
16 Oct	Lizard Scanner runs on the real scanner. Ring belongs to camera B. Light test: exposure is brightness. Pulse idea falls
16 Oct	Frame capture with per-picture stamps added (shared with Who are yoUSB?)
18 Oct	First wall test: red and green noise. “Vertical lines with staggered dots.” Stripe code; refit within 2 mm
19 Oct	First live test: streaks; neighbour check added; second calibration worse, cause unknown
19 Oct	Paused. Cleaning wins

Glossary

- **UVC (USB Video Class):** the standard every ordinary webcam uses.
- **Extension unit:** a numbered box of a camera maker’s own controls inside UVC.
- **Exposure:** how long the camera collects light for one picture. On the Lizard, also how long the light flashes.
- **Gain:** amplification of the picture inside the camera. Brighter picture, more noise, same flash.
- **Structured light:** shining a known pattern on something and measuring how it bends or shifts.
- **Baseline:** the gap between projector and camera. A bigger gap gives a bigger shift.
- **Stripe code:** our name for the beads’ wobble left after taking away the plain stripe.
- **K map, c:** the calibration’s numbers: how strongly distance turns into shift at each place in the picture, and where the lens sits.

References

1. M. Maruyama and S. Abe, “Range sensing by projecting multiple slits with random cuts”, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 15, 647–651 (1993).
2. J. Salvi, J. Pagès and J. Batlle, “Pattern codification strategies in structured light systems”, *Pattern Recognition* 37(4), 827–849 (2004). doi:10.1016/j.patcog.2003.10.002
3. J. Geng, “Structured-light 3D surface imaging: a tutorial”, *Advances in Optics and Photonics* 3(2), 128–160 (2011). doi:10.1364/AOP.3.000128
4. B. Freedman, A. Shpunt, M. Machline and Y. Arieli (PrimeSense), “Depth mapping using projected patterns”, US patent 8,150,142 (2012). https://patents.google.com/patent/US8150142

5. USB Implementers Forum, *Universal Serial Bus Device Class Definition for Video Devices* (UVC 1.5). https://www.usb.org/document-library/video-class-v15-document-set
6. Linux kernel, “The Linux USB Video Class (UVC) driver”. https://www.kernel.org/doc/Documentation/userspace-api/media/drivers/uvcvideo.rst
7. Creality, *CR-Scan Lizard troubleshooting guide* (via RobotShop). https://cdn.robotshop.com/media/C/C3d/RB-C3d-40/pdf/creality3d-cr-scan-lizard-luxury-3d-scanner-troublshooting.pdf
8. poita66, “Running Creality 3D Scanners (JM Studio) on Linux with Wine”. https://gist.github.com/poita66/cab19abacae3011454f1183d0e85aaeb
9. CNX Software, “Creality CR-Scan Lizard review: an easy-to-use 3D scanner” (2022). https://www.cnx-software.com/2022/11/22/creality-cr-scan-lizard-review-an-easy-to-use-3d-scanner/
10. Intel RealSense, librealsense `ds-private.h` (emitter and laser-power extension-unit controls, for comparison). https://github.com/IntelRealSense/librealsense/blob/master/src/ds/ds-private.h
11. EmixamPP, *linux-enable-ir-emitter* (finding infrared-emitter controls, and its warning about writing to unknown controls). https://github.com/EmixamPP/linux-enable-ir-emitter
12. T. Moń (desowin), *USBPcap capture format*. https://desowin.org/usbpcap/captureformat.html
13. M. Armstrong, *Lizard Scanner* (app, research notes, test reports and data). https://github.com/The-Dorkknight/lizard-scanner
14. M. Armstrong, *Who are yoUSB?* (USB recording and analysis toolkit). https://github.com/The-Dorkknight/who-are-yousb

---

Text and figures: CC BY-NC 4.0. Cover and poster art supplied by the author. Not affiliated with or endorsed by Creality. “CR-Scan Lizard” is Creality’s product name, used here only to say which device this is about.